PROJECT PROPOSAL

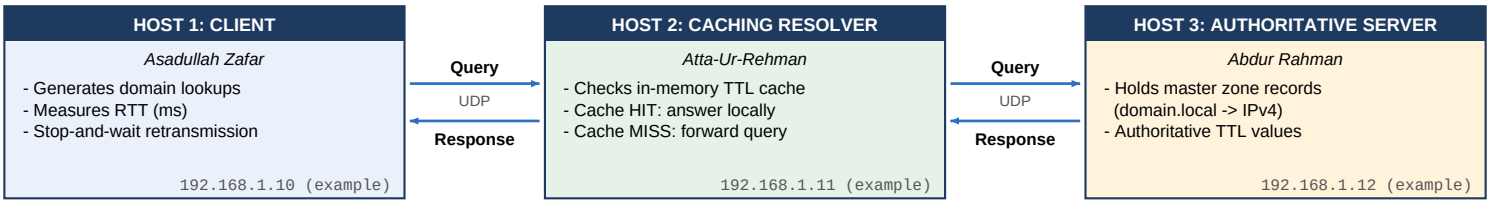
Design, Implementation, and Evaluation of a Multi-Host Networked Application

CS-327 Computer Communication Networks
Fall Semester 2026 | Group of 3
Selected Track: Option B — Mini DNS Service (MYDNS/1.0)

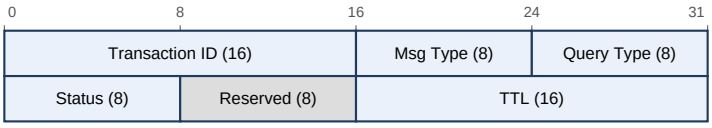
Member	Roll No.	Module / Host	Lead responsibilities
Asadullah Zafar	CS-24082	Host 1 — Client `src/client.py`	Client engine, microsecond RTT benchmarking suite, delay modeling
Atta-Ur-Rehman	CS-24092	Host 2 — Caching Resolver `src/resolver.py`	Resolver caching logic, thread-safe TTL engine, failure recovery
Abdur Rahman	CS-24098	Host 3 — Authoritative Server `src/authoritative_server.py`	Authoritative zone server, packet encoding specification, Wireshark encapsulation analysis

1. Project Overview & 3-Host Architecture

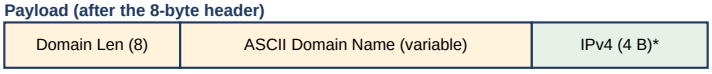
We will design and implement an original application-layer DNS protocol, **MYDNS/1.0**, directly over native UDP sockets with no third-party networking libraries. Like HTTP, it is just an agreed message format: each member implements one module against this protocol, so **the protocol is the only dependency between us** and everything else is modular and independent. The system runs on three independent endpoints (one laptop each):



2. Intended Protocol Specification (MYDNS/1.0)



Payload (after the 8-byte header)



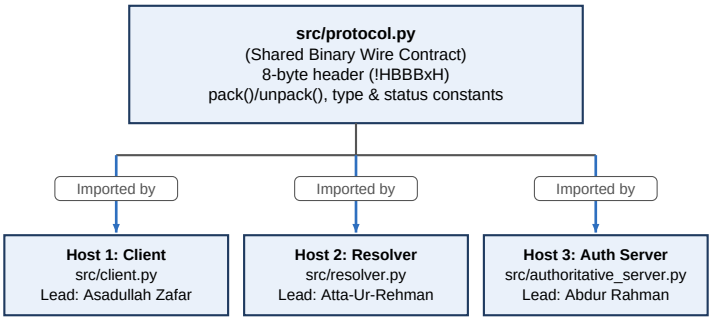
** Resolved IPv4 address is present in RESPONSE messages only.*

Fixed 64-bit (8-byte) binary header, packed with Python's `struct` format `!HBBBxH` (network byte order).

- **Transaction ID** (16 b, `!H`): matches request/reply pairs.
- **Msg Type** (8 b): `0x01` QUERY, `0x02` RESPONSE, `0x03` ERROR.
- **Query Type** (8 b): `0x01` TYPE_A (IPv4 lookup).
- **Status** (8 b): 0 SUCCESS, 1 NXDOMAIN, 2 SERVER_FAILURE, 3 FORMAT_ERROR.
- **Reserved** (8 b): alignment padding, `0x00`.
- **TTL** (16 b): validity of the record in seconds.
- **Payload:** 1-byte domain length + ASCII domain + 4-byte raw IPv4 (`socket.inet_aton`, response only).

3. Module Structure & Division of Work

Every module imports one shared file, `src/protocol.py` (the wire contract), so members can build and test independently.



- Each member owns exactly one module and one host.
- Integration only needs everyone to follow `protocol.py`, so nobody blocks anybody else.

4. Tools, Languages & Transport

- **Language:** Python 3 using only the standard library: `socket`, `struct`, `threading`, `time` (high-resolution RTT timing).
- **Analysis tool:** Wireshark, for packet capture and encapsulation analysis.
- **Transport: UDP (`SOCK_DGRAM`).** DNS needs minimal latency (no connection setup) and low byte overhead. Loss is handled at the application layer by client-side stop-and-wait retransmission (`socket.settimeout(1.5)`), avoiding TCP handshake cost.

5. Plan for Three Independent Hosts

- Each member runs their own module on **their own laptop**: Asadullah the client, Atta-Ur-Rehman the resolver (DNS server), Abdur Rahman the authoritative server.
- All laptops join one **mobile hotspot**. A hotspot creates its own local area network and assigns every laptop a unique IP address, so the three hosts talk to each other over real network links.
- The addresses in the diagram (`192.168.1.10/11/12`) are examples; actual IPs are whatever the hotspot assigns and are set in each module's config.

6. Experimental Benchmarks, Design Tensions & Sustainability

- **Performance benchmarks (R6):** measure RTT over at least 5 trials, comparing **cache hit vs. cache miss**, mapped to the nodal delay model: $T_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$
- **Failure analysis (R7):** simulate an unreachable authoritative host; verify resolver timeout handling and client-side error signaling.
- **Design tensions defended:** (1) caching freshness vs. traffic reduction (TTL tuning, stale-data risk); (2) reliability vs. latency (UDP application-layer retransmission vs. TCP connection establishment); (3) compactness vs. readability (binary packed structs vs. verbose text/JSON).
- **Sustainability analysis (Section 7):** a cache hit saves 50 bytes per query (100 bytes on a cache miss vs. 50 bytes on a hit), a 50% reduction in bandwidth and NIC energy per query.

Signatures

Asadullah Zafar (CS-24082) | Atta-Ur-Rehman (CS-24092) | Abdur Rahman (CS-24098)